

School of Information, Computer and Communication
Technology Sirindhorn International Institute of Technology
Thammasat University

Campus Lost and Found System

Final Report

Group Members

6522781762 Natthida Theepakornsukkasem
6522770807 Teerapat Charoensangsuwan
6522771045 Jakrapat Wongpradu
6522770708 Kawisara Premwinai
6522770724 Palinee Sadibhakdi
6522781770 Thatpol Wangkahat

Present to

Dr. Apichon Witayangkurn

DES424 Cloud-based Application
Development Semester 1 Academic Year 2025

Table of Contents

Overview and Background.....	1
Problem Statement.....	1
Key Users/Stakeholders.....	1
Functional Requirements.....	2
Nonfunctional Requirements.....	2
Programming Language.....	3
Cloud Technology and Components.....	3
Main Components.....	4
Micro Components & Security.....	5
Framework and Tools.....	5
Web Application Interface.....	6
Software Architecture.....	7
Subsystem Decomposition.....	7
Data Management.....	8
Boundary Conditions.....	8
Access Control and Security.....	9
Security : Broken Access Control Prevention.....	10
System Diagram Architecture on Cloud.....	10
Detailed Design.....	11
System Architecture.....	11
Workflow of the System.....	14
User Manual.....	15
Testing Report.....	27
User Acceptance Test (UAT).....	27
Robot Test.....	28
Links and Repository.....	35

Overview and Background

Lost items such as books, bags, and electronics are common on campus, yet traditional recovery methods—like posters or social media—are often unreliable and difficult to manage. To address this issue, the Campus Lost and Found System provides a centralized platform where students, staff, and visitors can report lost or found items, browse existing records, and receive updates from staff efficiently.

The system is fully deployed on Amazon Web Services (AWS), ensuring reliability, scalability, and security. It uses DynamoDB for fast serverless data storage, S3 for secure image handling, Cognito for user authentication, and AWS Lambda with API Gateway for backend processing. These services work together to deliver an organized and accessible solution for managing lost-and-found items across the campus.

Problem Statement

Losing personal belongings is a common issue on campus. Students and staff on campus frequently lose personal belongings, but current recovery methods are inefficient and unreliable. Lost items are often difficult to track, and individuals spend significant time searching without any guarantee of success. The existing manual process of matching lost and found items is slow and lacks automation, making it challenging to identify potential matches quickly. As a result, many items remain unclaimed, and users experience unnecessary frustration and inconvenience.

Key Users/Stakeholders

- SIIT Students
 - Students are the main users, able to see all items and report lost items and claim their belongings quickly.
- SIIT Staff and Faculty (Admin)
 - Staff and faculty can report items they lose or find and stay informed about campus lost-and-found updates.

Functional Requirements

- User Authentication
 - Secure login and registration for both *students* and *administrators* using email and password.
 - Role-based access control ensures that only authorized admins can manage or verify items.
- Reporting Items
 - Clients can report lost items by submitting item details such as name, type, date/time, and location.
 - Administrators can report found items with more details and descriptions and optional images before adding them into the system.
- Search and Filter
 - Clients and admin can search and filter items by keyword, date, location, and status filters make it easy to find items quickly.
- Match Suggestions
 - The system can automatically suggest potential matches between lost and found reports based on location, type, and date similarity. Admins can review and confirm these matches manually to maintain accuracy
- Admin Panel
 - Admins can view a summary of all items categorized by status and review usage statistics for oversight.
- Marking Item Status (Returned)
 - Admins can update an item's status to Returned after verifying owner information through the identification process.
- Owner Identification & Verification
 - Admin must submit an identification form when a student claims a found item, including name, role (student/other), student ID, phone number, and image upload.
 - Admins can view full owner information after verifying a claim
- Delete Item
 - Admin can remove incorrect or duplicate records.

Nonfunctional Requirements

Scope	Description
Performance	The system is designed to deliver fast response times, with quick search results and efficient image retrieval.
Scalability	As the number of students, staff, and visitors grows, along with an increasing volume of lost-and-found records, the system can scale to handle larger workloads. This includes supporting more simultaneous users, expanding storage, and maintaining consistent performance.

Reliability and Availability	The platform will be accessible 24/7 with minimal downtime, ensuring users can report or search for items at any time. Regular backups are maintained to prevent data loss.
Usability	The system provides an intuitive interface designed for both students and admin. Navigation between reporting, searching, and management functions should be simple and consistent across devices.
Data Integrity	The system ensures item records remain accurate and secure.

Programming Language

- **Frontend** : React.js
- **Backend** : Node.js

Cloud Technology and Components

AWS Service	Purpose	Details
API Gateway	API Entrypoint	Provides a secure, single point of access; routes all traffic to the backend logic.
Lambda	Serverless Compute	Executes all application code and hosts the sophisticated matching algorithm; scales automatically.
DynamoDB	NoSQL Database	Stores all reports and claim data; delivers single-digit millisecond performance at any scale.
Cognito	Authentication	Manages user identity and sign-in; integrates with the university's access system.
S3	Media Storage	Highly durable storage for item images and secure ID verification photos.
IAM	Security & Access	Defines and enforces permissions between all AWS components.
AWS Amplify Hosting	Deployment	Managed platform for rapid and reproducible deployment of the application code.

Main Components

The Lost & Found Cloud System is founded on a modern, robust serverless architecture leveraging key AWS services to maximize scalability, reliability, and security while minimizing operational overhead.

Frontend Hosting: AWS Amplify

AWS Amplify serves as the Continuous Integration/Continuous Deployment (CI/CD) and hosting platform for the React.js client application.

- **Function:** It automates the process of building and deploying the frontend code, distributing it globally via a Content Delivery Network (CDN), which provides users with fast, low-latency access to the application interface.
- **Operational Benefit:** Provides a fully managed hosting environment, eliminating the need to manage web servers or infrastructure scaling.

Authentication and Access Control: Amazon Cognito

Amazon Cognito is the core service managing all user identities, sign-up, sign-in, and role-based access to the system.

- **Secure Access Policy:** The `Login.js` component enforces a strict security requirement by utilizing a Pre-Sign Up Lambda Trigger to restrict registration and login to users with a valid university email domain (e.g., `@g.siiit.tu.ac.th`), ensuring a trusted user community.
- **Role Management:** Upon successful login, the application uses `Home.js` to check the user's **Cognito Group** (`admin` or `student`) to determine and render the correct administrative or student dashboard.

API Entrypoint: Amazon API Gateway

The Amazon API Gateway acts as the secure, unified interface for all communication between the client (Amplify) and the backend compute layer.

- **Function:** It exposes all backend business logic as secure RESTful API endpoints (e.g., the URL used in `ClientHome.js` to fetch all items). It manages request routing, rate limiting, and authenticates the user's Cognito token before invoking the relevant Lambda function.

Serverless Logic: AWS Lambda

AWS Lambda is the core compute service that executes all the application's business logic on-demand without the need for server provisioning.

- **Core Processes:** It powers critical endpoints like Report Lost (`reportlost.js`), Report Found (`reportfound.js`), and Item Deletion (`AdminDetail.js`), handling the logic of data formatting and database interaction.
- **Claim Verification Data Aggregation:** A dedicated Lambda function is responsible for processing requests from the `OwnerList.js` admin page. It fetches the list of outstanding claims, including the claimant's information (Name, ID, Tel), the claimed item's PK, and the associated S3 photo keys, consolidating this data for review by the administrator.

- **Item Status Management (Mark Returned):** Triggered by the `Identification.js` form after staff verification, this is a critical administrative function. The Lambda function executes a `UPDATE` operation on the DynamoDB item record, securely changing the item's `status` attribute to `RETURNED`, thus concluding the item's lifecycle and immediately updating the `AdminDashboard.js` statistics.
- **Matching Engine:** It hosts the automated matching logic that compares "Lost" reports against "Found" inventory based on categories and timestamps to suggest potential matches to administrators.

Database: Amazon DynamoDB

Amazon DynamoDB is employed as the primary, fully managed NoSQL database for structured data storage.

- **Performance:** Provides consistent, single-digit millisecond latency for all CRUD (Create, Read, Update, Delete) operations, ensuring that item lists and dashboards load instantly.
- **Data Structure:** It stores all essential application data, including user reports, item records, claim records, and status (LOST, FOUND, RETURNED) used for filtering.

Media Storage: Amazon S3

Amazon Simple Storage Service (S3) is utilized for secure, durable, and highly available object storage, specifically for files.

- **Usage:** S3 stores uploaded media, including item images and, most critically, the **ID verification photos** required for claims.
- **Claim Verification Access:** The `OwnerList.js` and `Identification.js` components generate secure, temporary URLs from S3 to retrieve and display the sensitive photo proof submitted by users claiming an item, centralizing and protecting this evidence with access control.

Micro Components & Security

These components handle security permissions, administrative workflow automation, and application deployment.

Identity & Access Management: AWS IAM

AWS Identity and Access Management (IAM) is fundamental to the system's overall security model.

- **Function:** It is used to define and manage permissions and roles, ensuring that every AWS component (like Lambda) only has the exact access required to interact with other services (like reading/writing to DynamoDB and S3).
- **Security Principle:** Enforces the principle of least privilege, preventing a compromised service from accessing data or services outside its defined operational scope.

Deployment: AWS Amplify Hosting (CI/CD)

The deployment strategy focuses on providing a managed, scalable, and fully-automated CI/CD (Continuous Integration/Continuous Deployment) environment for the React frontend.

- **Function:** Amplify Hosting offers a high-level, Git-based platform. It connects directly to the GitHub repository and automatically triggers a new build and deployment whenever changes are pushed to the main branch. It deploys the static React frontend build to a global Content Delivery Network (CDN) for high performance.
- **Benefit:** This approach is ideal for rapid iteration, as it handles the entire build and deployment process without requiring management of the underlying infrastructure. It securely connects to the existing AWS backend (API Gateway, Lambda) using the environment variables set in the Amplify console.

Framework and Tools

Purpose	Tools / Frameworks
Project Management	Jira
Version Control	GitHub
Code Editor	VS Code
Frontend Development	React.js
Backend Development	Node.js (Lambda)
Database Management System	DynamoDB
Deployment	AWS Amplify Hosting
Software Testing	Robot Framework

Web Application Interface

- **User Application Interface**

User interface consists of main page which show all item, item detail , and report lost form

The main page features a search bar at the top with a magnifying glass icon. Below it are filter buttons for 'Filter', 'Category', 'Date', and 'Location'. A section titled 'All Item >' displays a grid of six item cards. Each card has a 'Topic' header, a 'Title' field, and a 'Description' field.

The 'Item Detail' page displays the following information: Item : Wallet, Title : Brown Leather Wallet, Item Description : Contains student ID and cash., Date : 10 Oct 2025, Time : 14.30, and Location : Bangkadi Campas. A 'Back' button is located at the bottom right.

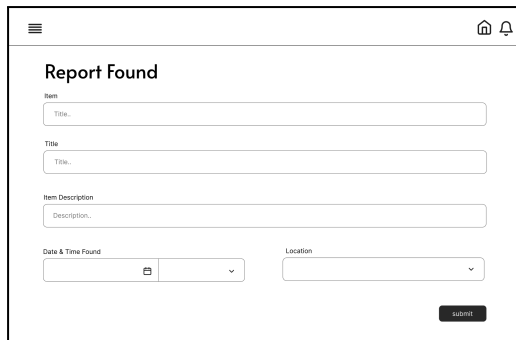
The 'Report Lost' form includes an 'Item' field with a placeholder 'Title...', a 'Date & Time Lost' field with a calendar icon, and a 'Location' field. A 'Submit' button is at the bottom right.

- **Admin Application Interface**

The admin interface consists of a main page that shows a summary of all items categorized, an item list page, a report found form, an item detail page, and an identification page for returned belongings.

The 'Summary' page shows four statistics: Lost (4), Found (5), Matched (3), and Verified (2). Below these is a 'Recent Activities Table' with columns for ID, Item, User, Status, Date, and Time.

The 'All Item List' page features a search bar and filter buttons. It displays a grid of item cards, each with a 'Topic', 'Title', and 'Description' field. A sidebar on the left contains links for 'Report' and 'Item list'.



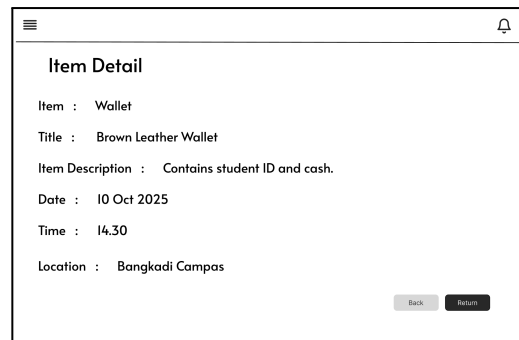
Report Found

Item
Title:

Title
Title:

Item Description
Description:

Date & Time Found Location



Item Detail

Item : Wallet

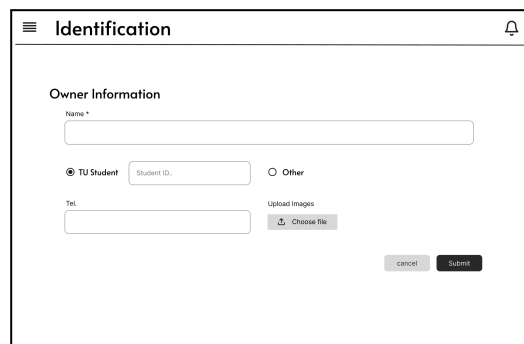
Title : Brown Leather Wallet

Item Description : Contains student ID and cash.

Date : 10 Oct 2025

Time : 14.30

Location : Bangkadi Campas



Identification

Owner Information

Name *

☒ TU Student ☐ Other

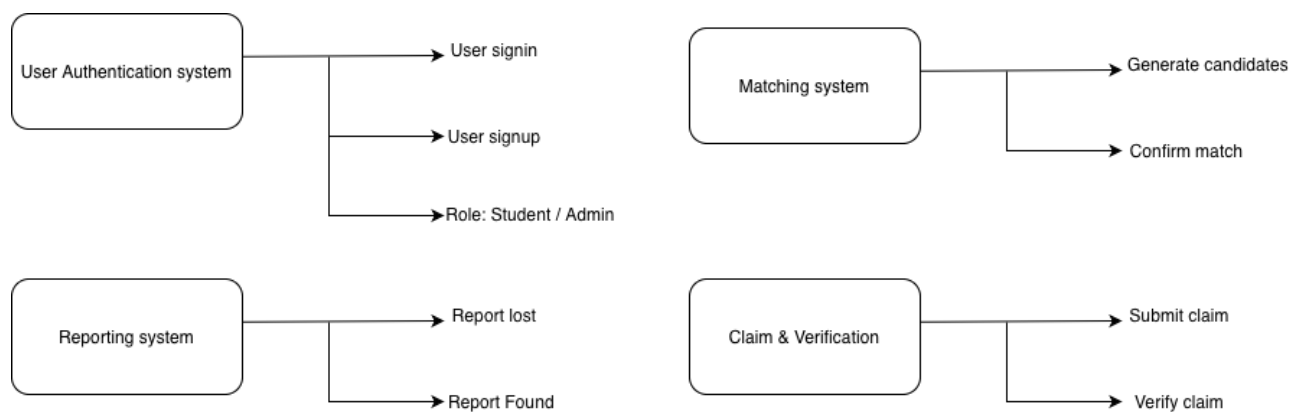
Tel

Upload Images

Software Architecture

Subsystem Decomposition

The diagram below shows the subsystems in our campus lost and found system web application.



Data Management

Persistent data

- **Data Storage** : Using DynamoDB for system records, it stored all structured information, including lost item reports, found item entries, user profiles, match results, and claim histories. Using S3 for media, S3 handles all uploaded images, such as item photos.
- **Data Integrity** : All records include unique identifiers, timestamps, and status. Claim logs and ID verification images are stored securely to ensure authenticity and prevent tampering during the claim and return process.

Boundary Conditions

Initialize

When the Lost and Found System starts, it initializes core services like the database and cloud resources, checks the readiness of DynamoDB tables and API Gateway endpoints, and loads configuration data for user roles and access permissions. The frontend connects to the backend services, conducts a health check, and verifies user login for reporting lost or found items.

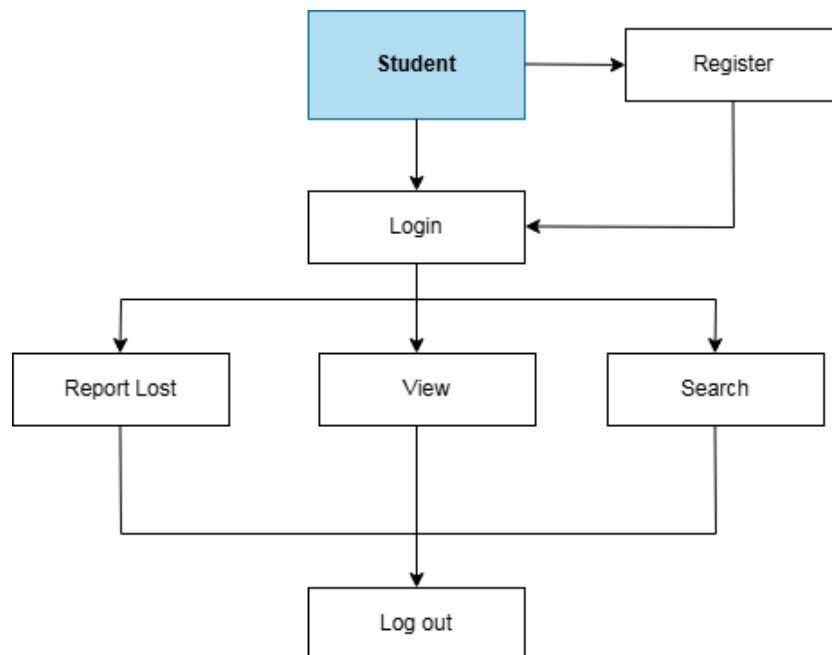
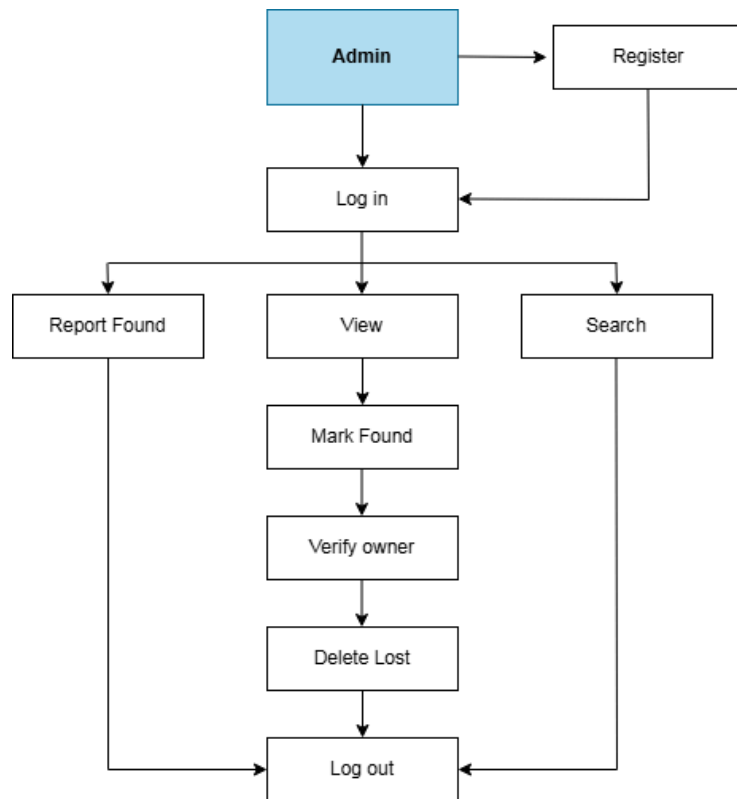
Termination

During shutdown or maintenance, the system gracefully terminates ongoing operations, blocks new requests, ends user sessions, and backs up recent data to prevent loss. Cloud resources like Lambda functions and APIs are paused, with a maintenance message displayed to inform users of temporary unavailability.

Failure

If a system failure occurs, it automatically detects and manages the issue to minimize disruption. Users receive clear error messages for authentication errors, network failures, or database issues, allowing retries. Failed API calls are logged and retried, while critical errors alert administrators for quick recovery. Backup and monitoring systems ensure data integrity and system reliability.

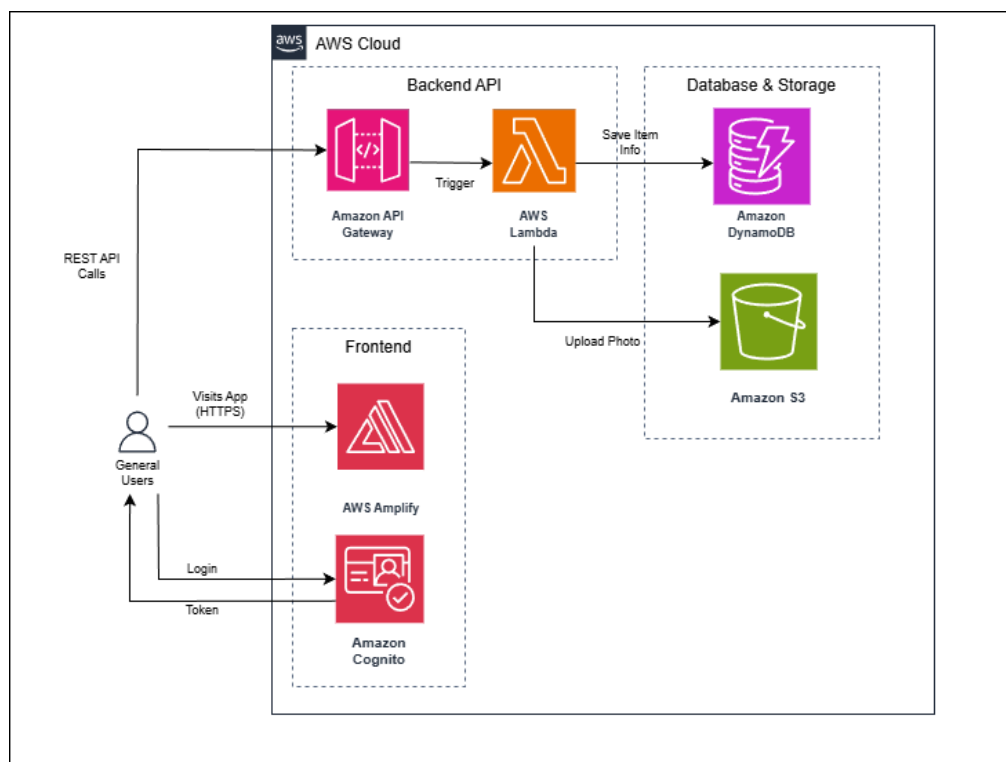
Access Control and Security



Security : Broken Access Control Prevention

Our system prevents broken access control through multiple integrated layers of security across the frontend, backend, and AWS infrastructure. On the client side, React route guards check the user's Cognito group and prevent students from accessing staff or admin pages, blocking direct URL access and unauthorized navigation. AWS Cognito adds server-side enforcement by issuing signed ID tokens that cannot be forged or modified, storing each user's role securely in Cognito Groups (student, staff, admin), and rejecting any request with an invalid or missing token. API Gateway and Lambda provide a second backend barrier by validating the caller's token on every request and checking user roles before allowing sensitive actions such as deleting items or verifying claims. Finally, DynamoDB access logic ensures that students can only view their own reported items, while staff and admins can access all records. Together, these combined controls ensure that even if a user manipulates the browser, edits JavaScript, or attempts to directly call backend APIs, they still cannot access data or functions beyond their assigned permissions.

System Diagram Architecture on Cloud

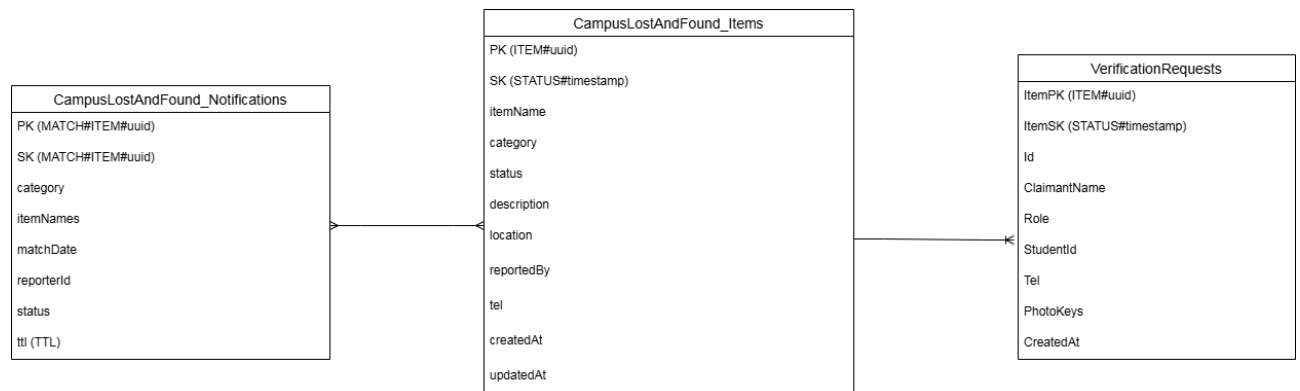


The Campus Lost and Found System utilizes a robust serverless architecture on AWS starting with the React frontend hosted on AWS Amplify for fast global delivery. User identity and access are secured through Amazon Cognito while all application requests are routed via Amazon API Gateway to trigger AWS Lambda functions. These serverless functions execute the core business logic by efficiently storing structured item data in Amazon DynamoDB and managing image uploads in Amazon S3 to ensure a highly scalable and maintenance-free infrastructure.

Detailed Design

ER Diagram

In this section, use DynamoDB for our database system which is a non-relational database. The equivalent of an ER diagram in a non-relational database is shown on the following diagram.



System Architecture

The Campus Lost and Found System is developed as a cloud-based, serverless web application using Amazon Web Services (AWS). The architecture follows a three-layer design consisting of the presentation layer, application layer, and data layer. The frontend is implemented using React.js, while the backend relies on API Gateway and AWS Lambda to process all application logic. All persistent data is stored in DynamoDB, with image files saved securely in Amazon S3.

Frontend Logic (React.js UI, API calls)

The frontend is built using React.js as a single-page application (SPA). It provides separate interfaces for student users and system administrators. React Router is used to support navigation between pages such as the home page, item list, item detail, lost-item reporting page, admin dashboard, owner identification list, and matching list.

The user interface handles all client-side processing, including form validation, state management, and dynamic rendering of content based on the user's role. When a user performs an action—for example, submitting a lost item report or filtering items by category—the frontend sends an HTTP request to the backend through API Gateway. All requests and responses use JSON over HTTPS to ensure security and integrity.

Key frontend responsibilities include:

1. **User Authentication and Role Handling**

Users can register and log into the system using their email and password. After authentication, the frontend stores the issued token and uses it to differentiate between student users and administrators. This determines which screens and actions are accessible.

2. **Lost and Found Item Interaction**

The frontend displays available lost and found items retrieved from the backend. Students are able to report lost items, search and filter items, and view item details. Administrators use the same interface to access additional options for managing item status, reviewing matches, and verifying owner claims.

3. **Form Submission and Validation**

When users submit forms—such as reporting a lost item, reporting a found item, or completing the owner identification form—the frontend validates required fields before sending the data to the backend. If an image is included, it is uploaded as part of the submission process.

4. **Integration with Backend APIs**

The frontend interacts with the following backend API Gateway endpoints:

- **GET /items** – Retrieve all lost, found, and returned items.
- **POST /items/found** – Submit a new found item report (admin only).
- **POST /saveVerificationData** – Save owner identification form details.
- **POST /getVerificationUploadUrl** – Generate a pre-signed S3 URL for image uploads.
- **GET /admin/ownerlist** – Retrieve pending owner verification requests.
- **POST /deleteItem** – Delete an item record from the system (admin).
- **POST /deleteItemV3** – Updated deletion endpoint for admin.

These endpoints provide the core backend functionality that supports both student operations (reporting and searching for items) and administrative operations (verification, moderation, and system management).

5. All requests requiring user privileges include an authentication token in the header.

6. **Feedback and Error Handling**

The UI provides feedback through toasts, alerts, and loading components. Errors returned by the backend—such as invalid input, unauthorized access, or network issues—are displayed to the user through appropriate messages.

Backend Logic (Lambda functions, API Gateway endpoints)

The backend is implemented using AWS Lambda in conjunction with Amazon API Gateway. The architecture follows a serverless approach, allowing the system to scale automatically based on usage and eliminating the need for server provisioning or maintenance.

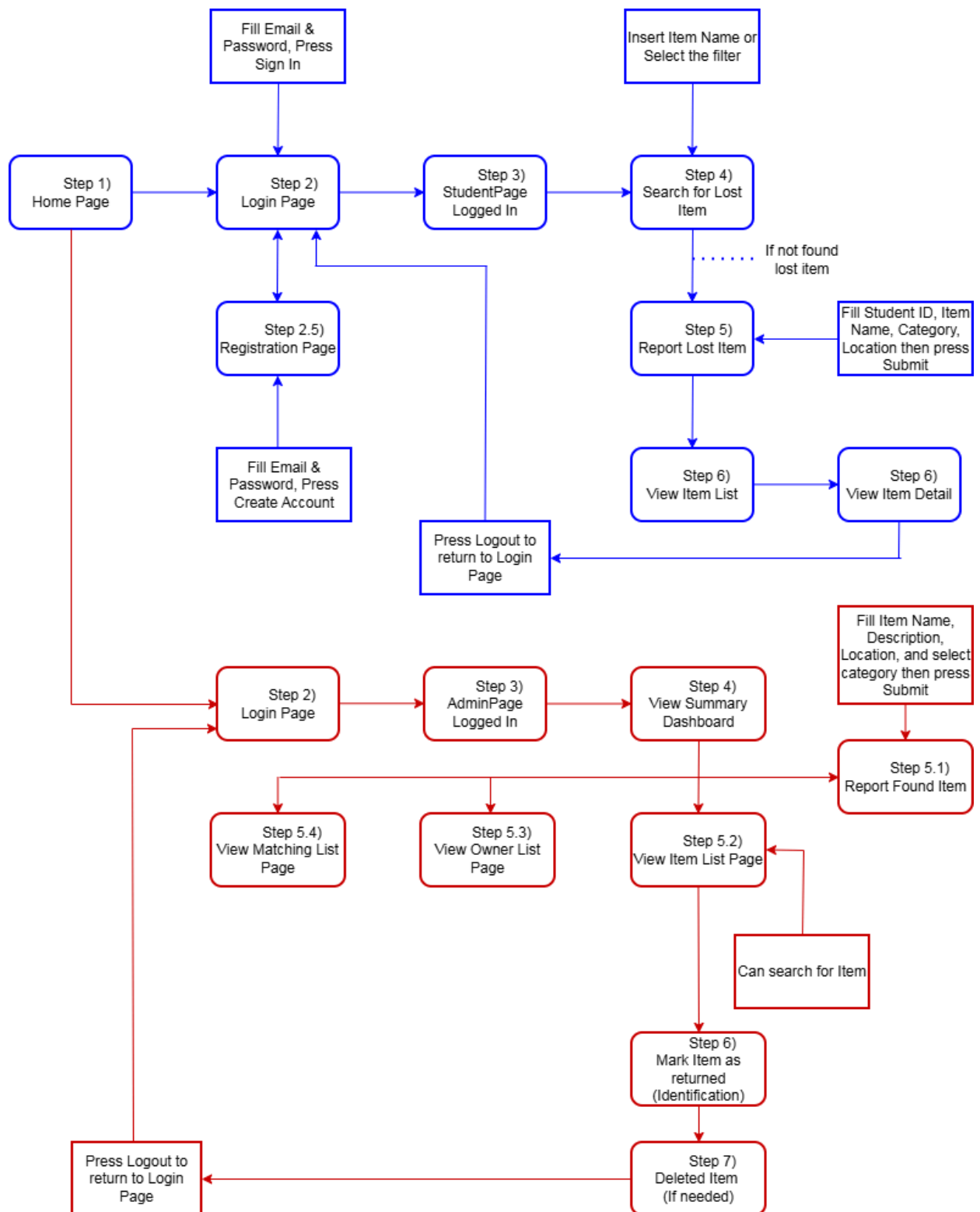
API Gateway serves as the main entry point for all external requests originating from the React frontend. It defines RESTful routes corresponding to application functions, including retrieving items, submitting new item reports, processing owner identification forms, reviewing match suggestions, updating statuses, and deleting items. Each route is mapped to a specific Lambda function responsible for executing the associated logic.

The backend logic includes the following components:

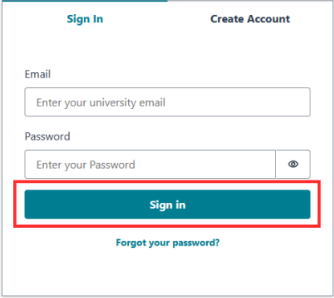
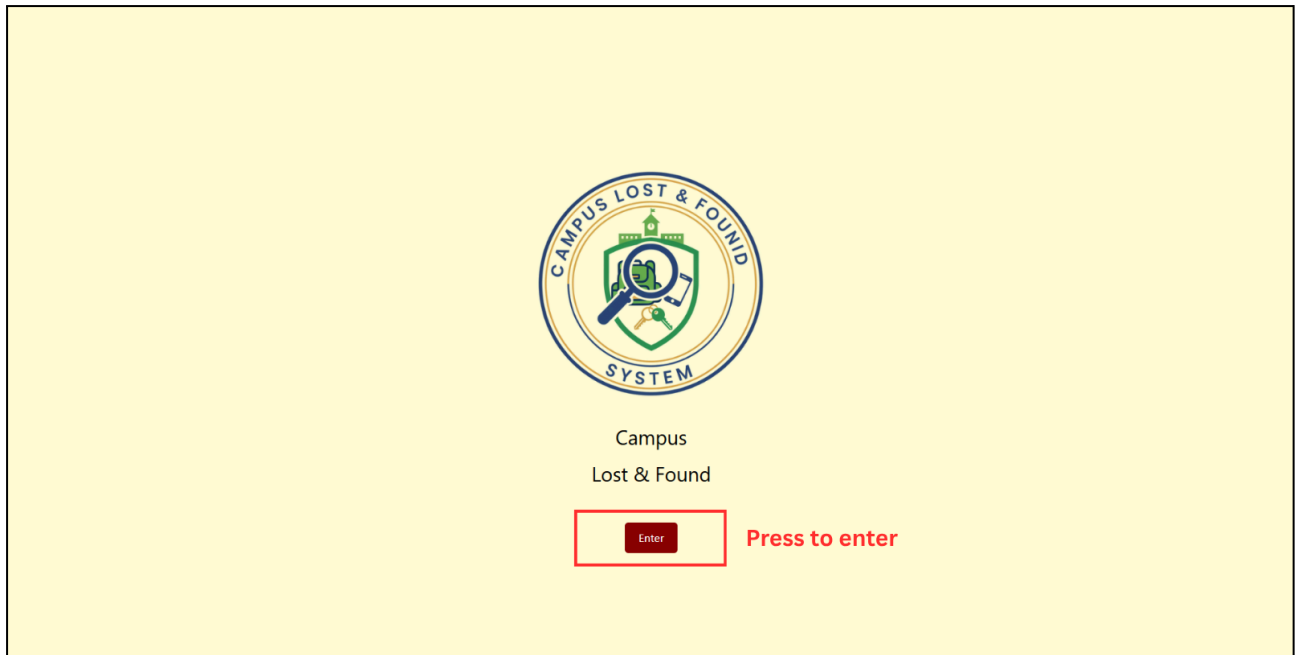
- 1. Report Lost Item Function (Student Only)**
This Lambda function validates user input, generates a unique item identifier, and stores the lost item data in DynamoDB.
- 2. Report Found Item Function (Admin Only)**
Administrators can report found items using this function. It handles additional metadata such as item description, category and location. The function may also run initial logic to check for potential matches.
- 3. Retrieve Item List Function**
This function scans or queries the DynamoDB table to retrieve lists of items. It supports optional filters such as category, location, and status, enabling both students and administrators to perform targeted searches.
- 4. Retrieve Item Details Function**
This function fetches complete details of a specific item, including metadata, and status history.
- 5. Dashboard Summary Function (Admin Only)**
For administrators, this function calculates operational statistics used in the dashboard, such as total items, total lost/found/returned items, number of items reported today, and category breakdowns.
- 6. Owner Verification Functions (Admin Only)**
These functions retrieve pending owner identification requests and allow administrators to update item status and record owner information after verifying the owner. Status updates are saved directly to DynamoDB.
- 7. Matching Function**
This function retrieves automatically generated lost-found match suggestions, allowing administrators to review it manually.
- 8. Delete Item Function**
This function enables administrators to remove incorrect or duplicate records. It performs authorization checks before deleting the item in DynamoDB.

Each Lambda function is assigned an AWS IAM role containing only the permissions required for its operation. This includes specific permissions for accessing DynamoDB tables, uploading or retrieving images from S3, and writing execution logs to CloudWatch.

Workflow of the System



User Manual



Sign in

Create Account

Email

Enter your university email

Password

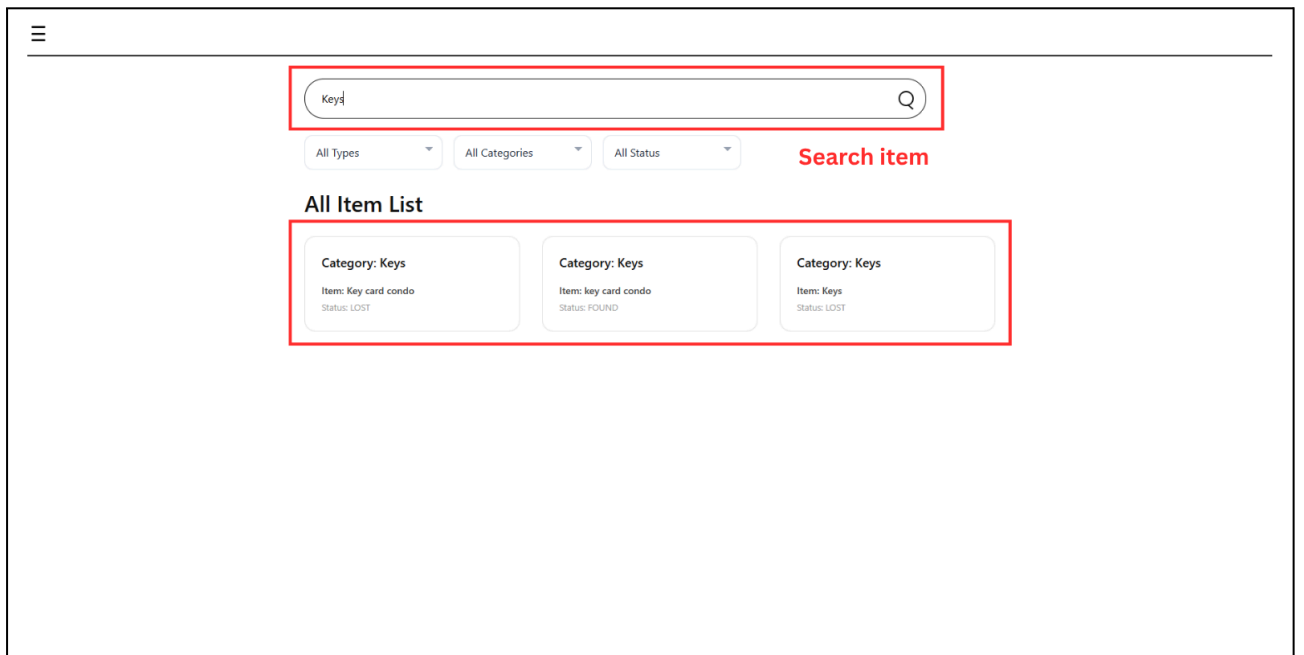
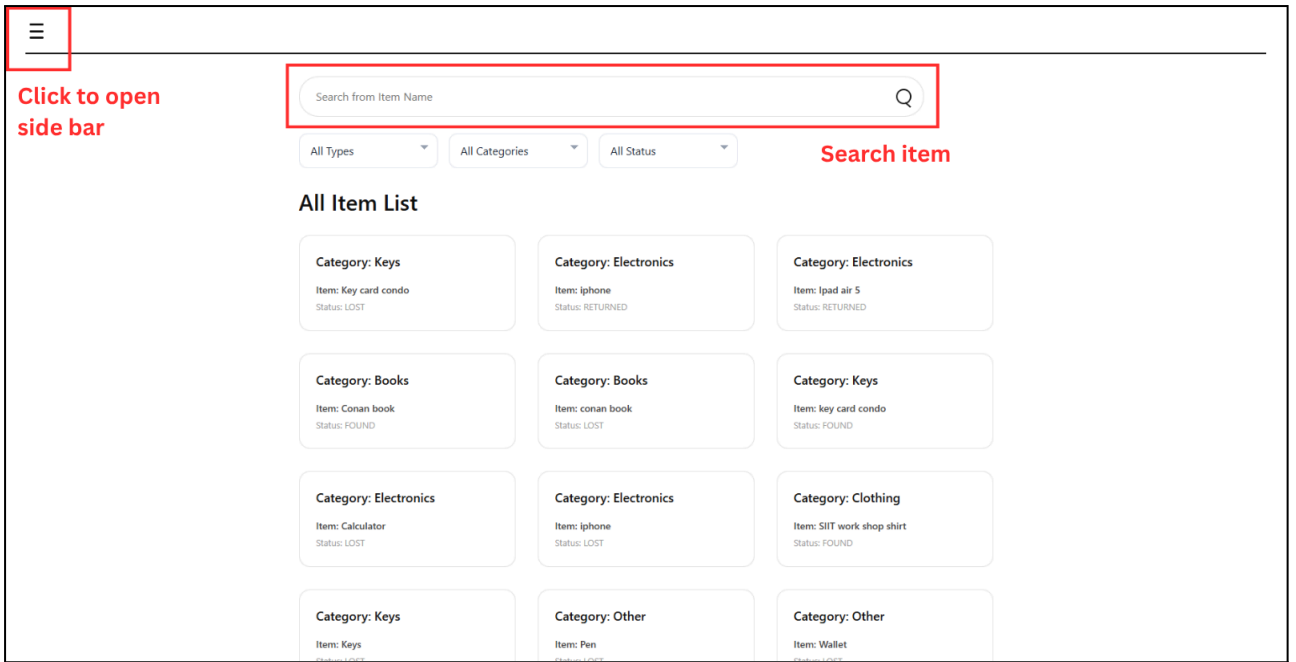
Enter your Password

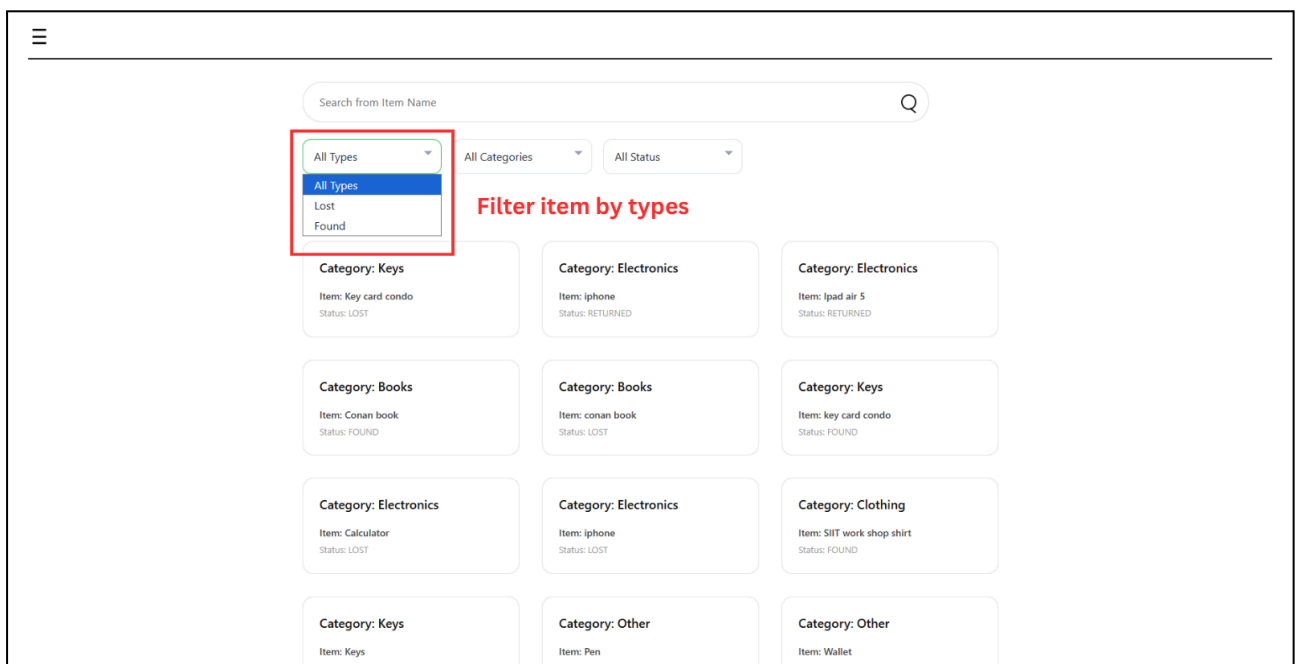
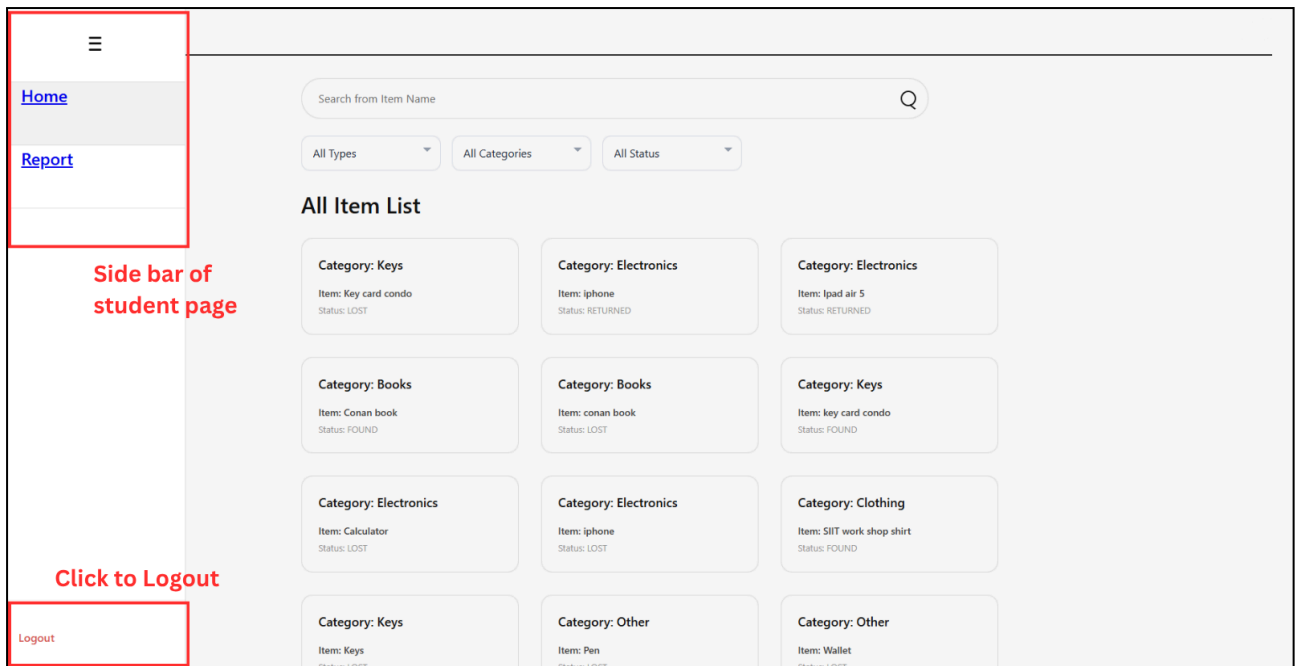
Sign in

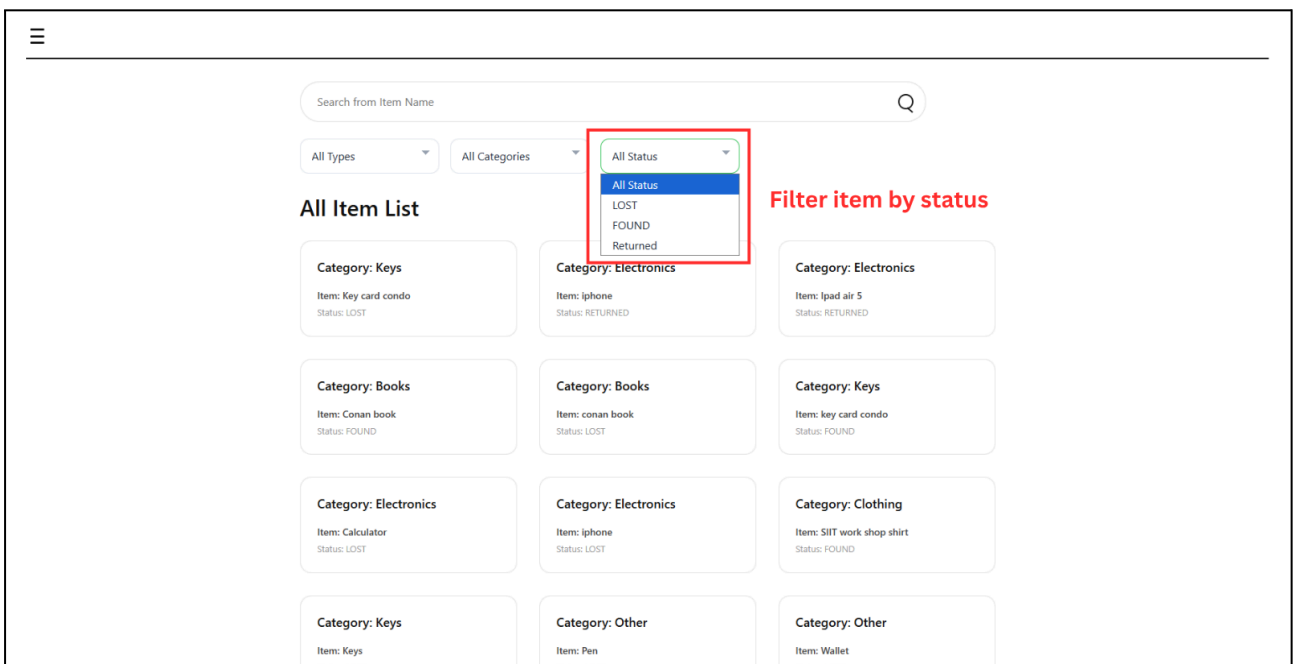
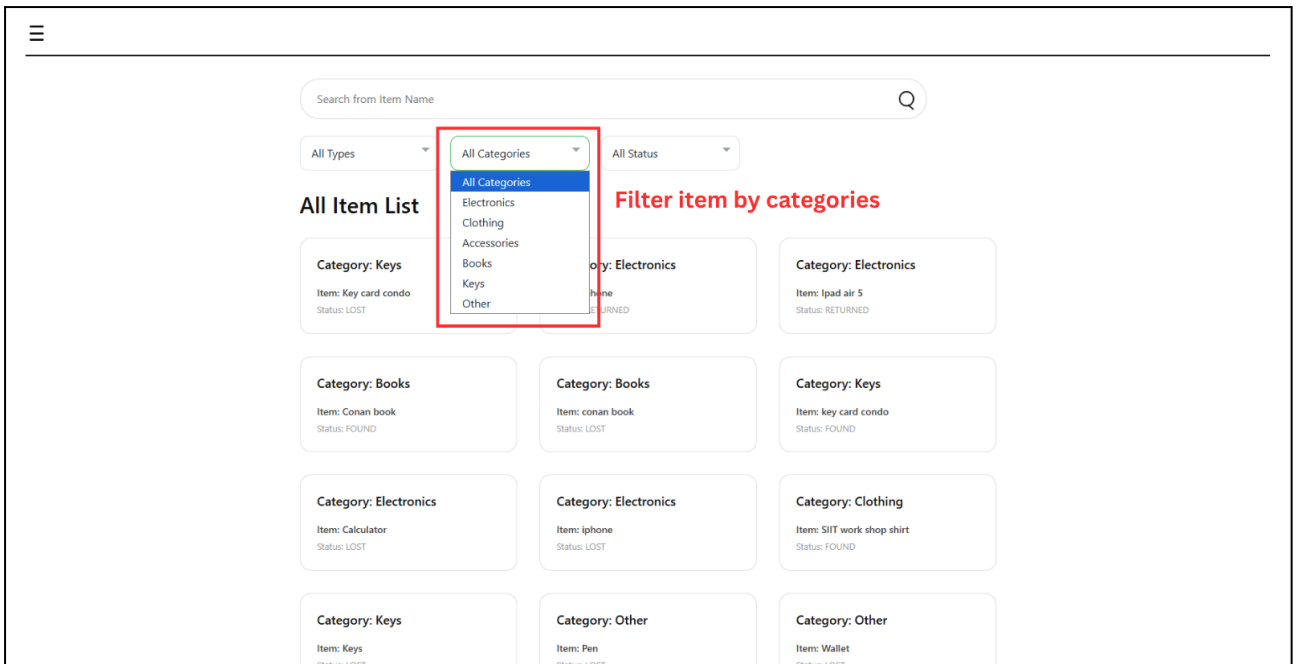
Forgot your password?

Fill the email & password,
and press Sign in to login

The image shows a white rectangular form with a light blue border. At the top, there are two tabs: 'Sign in' and 'Create Account'. Below the tabs, there are two input fields: 'Email' and 'Password'. The 'Email' field has a placeholder text 'Enter your university email'. The 'Password' field has a placeholder text 'Enter your Password' and a toggle icon. Below the input fields is a red rectangular button with the text 'Sign in'. Below the button is a link 'Forgot your password?'. To the right of the form, the text 'Fill the email & password, and press Sign in to login' is written in red.







Search..

All Types

All Categories

All Status

All Item List

Category: Keys

Item: Key card condo

ID: ITEM#9a073232-6acd-42cc-b1cf-7b440bbdb564

Status: LOST

Category: Electronics

Item: iphone

ID: ITEM#d33f212-f608-4341-aa9f-5c52e9350f8

Status: RETURNED

Category: Electronics

Item: Ipad air 5

ID: ITEM#6afbe372-a9cd-477d-aa02-7866a3b515dd

Status: RETURNED

Category: Books

Item: Conan book

ID: ITEM#975852f9-404e-4068-96d8-2e05ea7663ac

Status: FOUND

Category: Books

Item: conan book

ID: ITEM#9a8b307e-88a2-4c40-b3de-51bda3114ac2

Status: LOST

Category: Keys

Item: key card condo

ID: ITEM#16c1460a-fa7a-441c-8a19-bee195285113

Status: FOUND

Category: Electronics

Item: Calculator

ID: ITEM#444ef244-45a2-43f9-ae62-af5a1c0d8ab6c

Status: LOST

Category: Electronics

Item: iphone

ID: ITEM#e6c7bdfa-bcb4-472f-9c62-03f929c005e0

Status: LOST

Category: Clothing

Item: SIIT work shop shirt

ID: ITEM#0aaddcde-39a8-40d8-a151-0851d68c963

Status: FOUND

All items on the list show all the reports are reported. Click to show item detail.

Key card condo Details

Item Status:

LOST (Lost)

Item Name:

Key card condo

Category:

Keys

Date Lost/Found:

Nov 16, 2025

Time Lost/Found:

12:00

Location:

BKD

Reported By ID:

6522779999

Click to go back to previous page

Back

Report Lost Item

Tell us what you lost.

Fill and select the information

Student ID *
6522770708

Item Name *
Airpod Pro 2

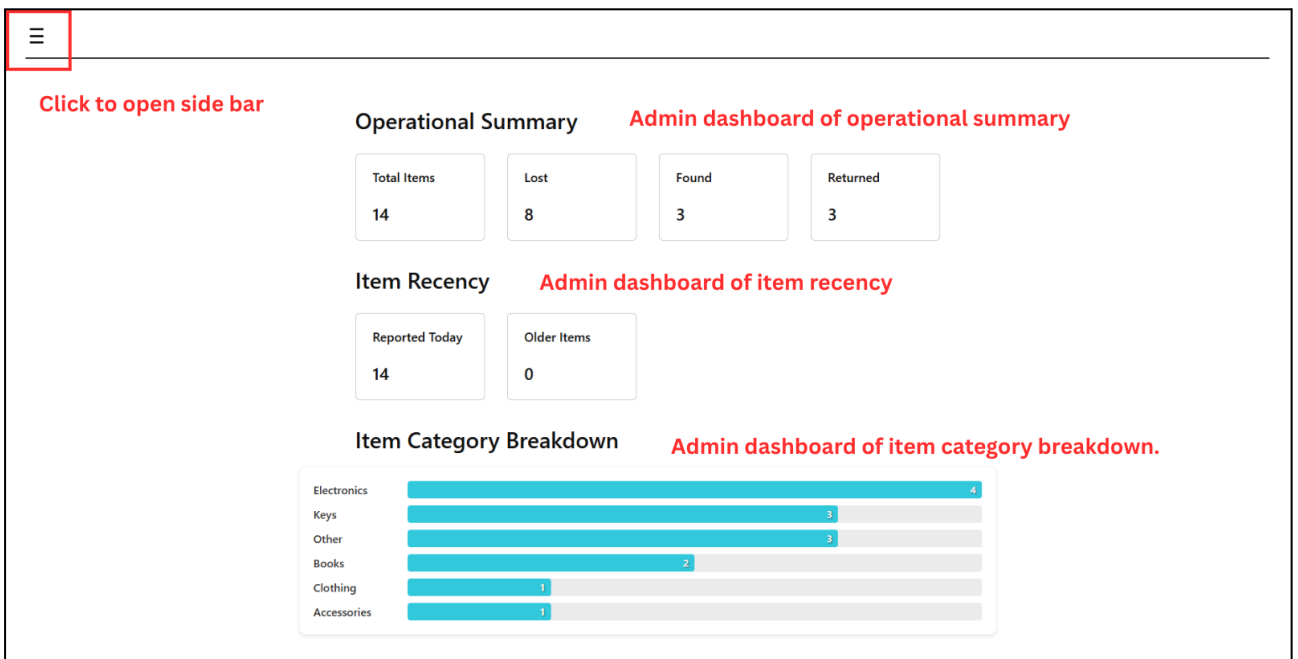
Category *
Electronics

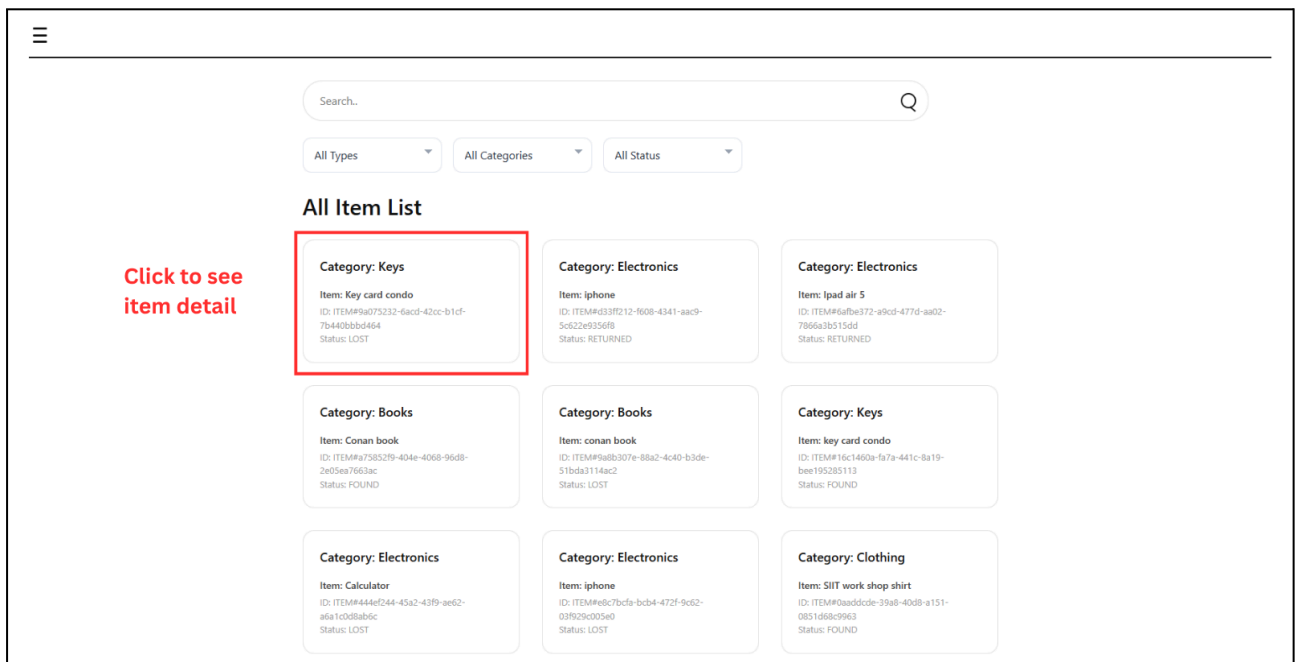
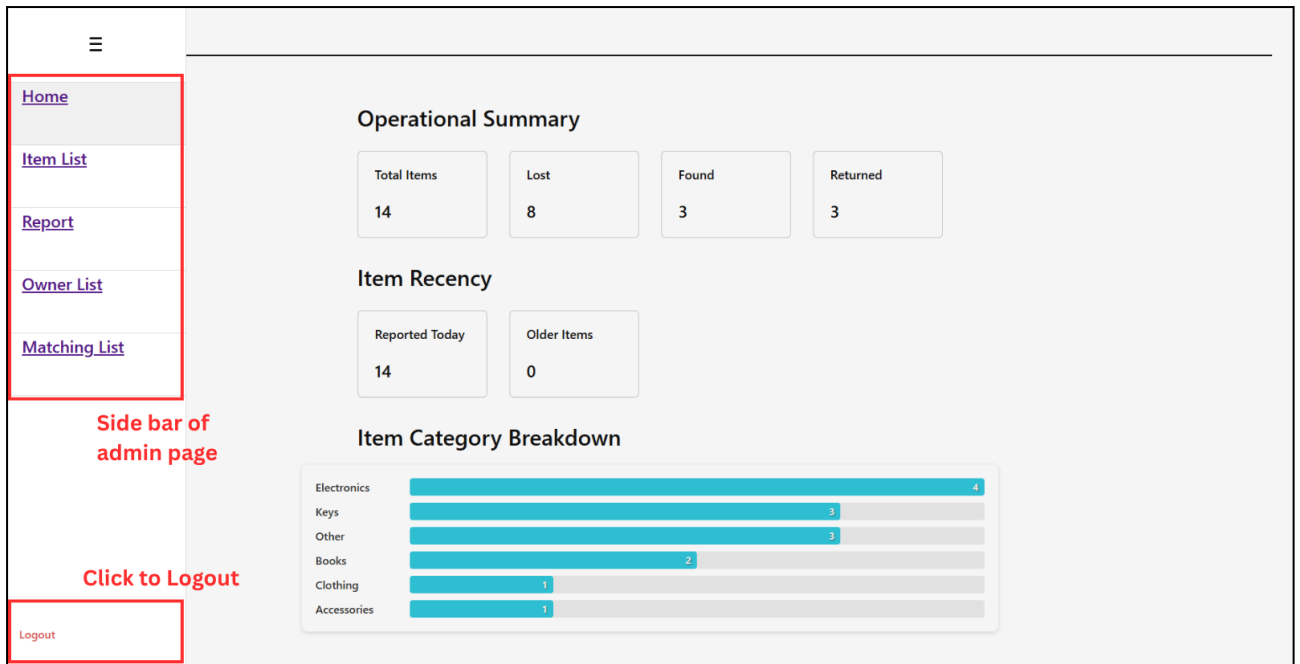
Date and Time Lost *
11/17/2025 12:45 AM

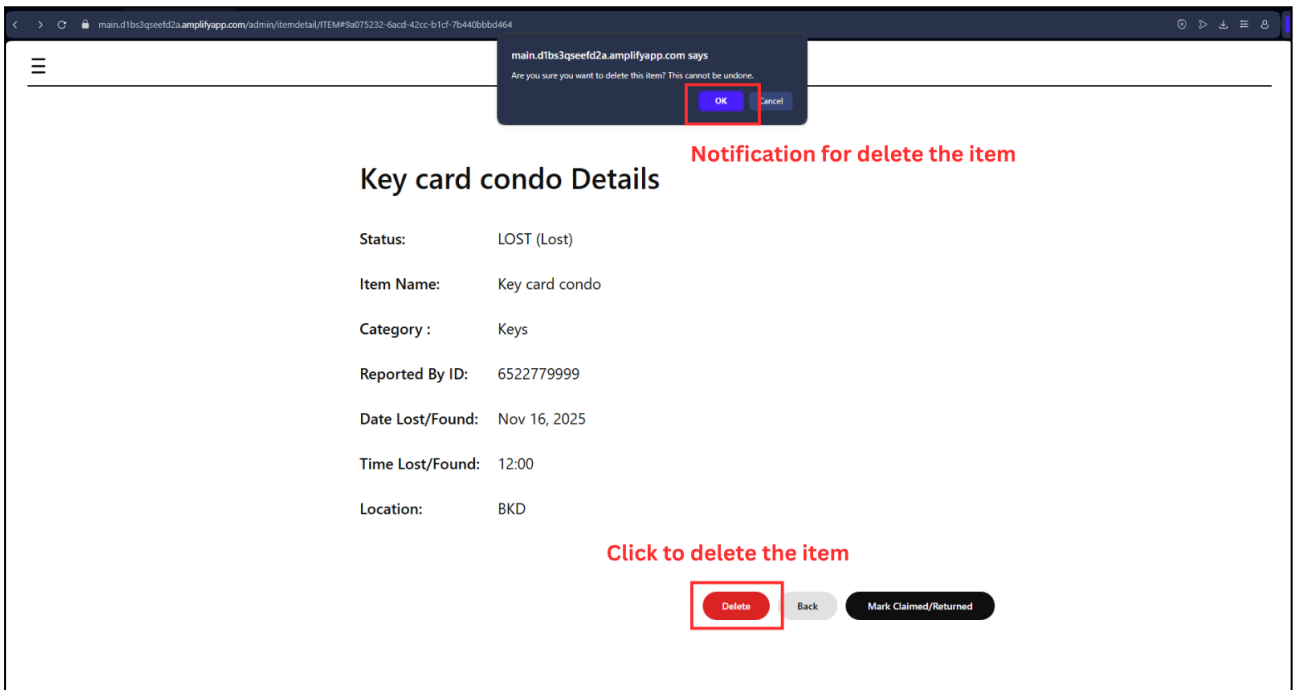
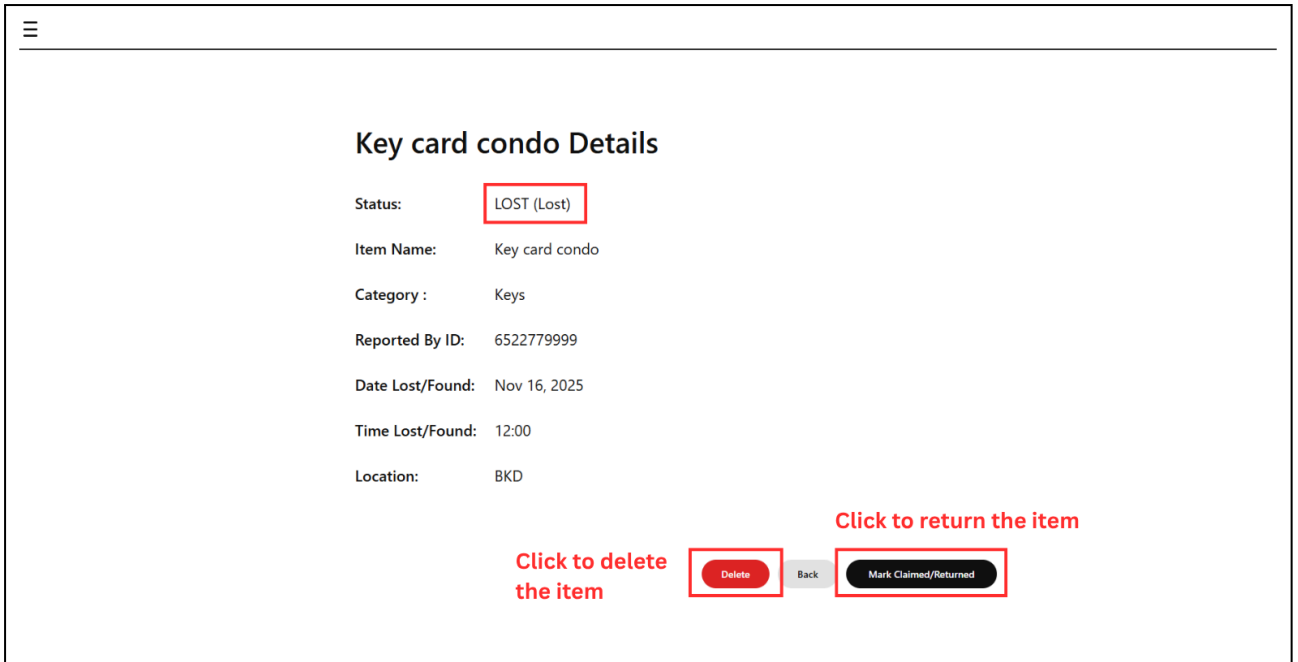
Location Lost *
SILT Main Building

Submit Report Cancel

**After fill in information,
press the submit button**







Finalize Claim / Return

Item Details

Item: **Key card condo** (9a075232...)
Reported as: **Keys**. Current Status: **LOST**

Owner Information

Fill the information

Name *

Owner's name

☒ SIIT Student

Student ID...

☐ Other

Tel.

Phone number

Upload Images

Choose file

cancel

Submit

Choose the images that want to upload

Click submit button to send the identification report

Finalize Claim / Return

Item Details

Item: **Key card condo** (9a075232...)
Reported as: **Keys**. Current Status: **LOST**

Owner Information

Confirm marking item **Key card condo** as **RETURNED**?

Yes

No

Click Yes to confirm to mark the item as RETURNED

24

iphone Details

Status:

RETURNED (Found)

If status is 'Found',
the Mark Returned
button is not show

Item Name:

iphone

Category :

Electronics

Reported By ID:

admin

Date Lost/Found:

Oct 2, 2025

Time Lost/Found:

03:22

Location:

SIIT roof top

Delete

Back

Report Found Item

Tell us what you found.

Fill and select the information

Item Name *

Airpods Pro 2

Description *

Airpods pro 2 with red case and heart keychain

Category *

Electronics

Date and Time Found *

11/18/2025 02:01 AM

Location Found *

SIIT Main Building

Office Notes (Optional)

Internal notes on item condition or storage location.

Submit Report

Cancel

After fill in information,
press the submit button

Owner Verification Lists (3)

Review pending claims to verify the identity of the user attempting to claim a lost item.

Jakrapat Wongpradu

Request ID: req-176347270828...

Student ID: 6522771045

Tel: 0830145341

Item ID: ITEM#9846b46f-313d-4644-9d74-6e41f31a9c4

Submitted: 11/18/2025, 8:31:48 PM

Photo Proof Attached

Open Review

Palinee

Request ID: req-176347298288...

Student ID: 6522770724

Tel: 0909009000

Item ID: ITEM#6afbe372-a9cd-477d-aa02-7866a3b515dd

Submitted: 11/18/2025, 8:36:22 PM

Photo Proof Attached

Open Review

Test

Request ID: req-176347866873...

Student ID: 29939393

Tel: 727272727

Item ID: ITEM#d33f212-f608-4341-aa09-5c62a9356f8

Submitted: 11/18/2025, 10:14:28 PM

Photo Proof Attached

Open Review

Click the card to see more owner details

Review Verification Request

Request ID: req-1763472708284-82185

Claimant Name

Jakrapat Wongpradu

Student ID

6522771045

Role

student

Telephone

0830145341

Item ID

ITEM#9846b46f-313d-4644-9d74-6e41f31a9c4

Submitted On

11/18/2025, 8:31:48 PM

Photo Proof

Key: verification/1763472704109-78C2868A-8008-4DC0-AA8A-0D8D34AC6A19.jpg



Show the detail of verification

26



Potential Match Review

Pending matches across all categories.

Lost Item: key card condo
Found Item: Key card condo
Lost Item Reported by User ID: 6522779999
Category: keys
Item ID: ITEM#16c1460a-fa7a-441c-8a19-bee195285113
Matched Item ID: ITEM#9a075232-6acd-42cc-b1cf-7b440bbbd464

Lost Item: iphone
Found Item: iphone
Lost Item Reported by User ID: 6522771045
Category: electronics
Item ID: ITEM#d33ff212-f608-4341-aac9-5c622e9356f8
Matched Item ID: ITEM#e8c7bcta-bcb4-472f-9c62-03f929c005e0

Lost Item: conan book

When lost and found items match,
it will be displayed in the potential match review

Testing Report

User Acceptance Test (UAT)

Campus Lost And Found System User Acceptance Test			
	Acceptance Requirements	Test Results	
No	Student page	Accept/Reject	Test Date
1	User can signin/signup as student.	TRUE	18/11/25
2	User can see all item list and item detail.	TRUE	18/11/25
3	User can search and filter item.	TRUE	18/11/25
4	User can report lost item.	TRUE	18/11/25
5	User can logout.	TRUE	18/11/25
	Admin page		
6	Admin can signin/signup as admin.	TRUE	18/11/25
7	Admin home page shows operational summary, item recency, item category breakdown.	TRUE	18/11/25
8	Admin can see all item list and item detail.	TRUE	18/11/25
9	Admin can delete item.	TRUE	18/11/25
10	Admin can search and filter item.	TRUE	18/11/25
11	Admin can report found item.	TRUE	18/11/25
12	Admin can see owner information list.	TRUE	18/11/25
13	Admin can see potential match review.	TRUE	18/11/25

Robot Test

LostAndFoundWebTests.robot

*** Settings ***

Documentation End-to-End UAT Workflow: Student Report, Admin Return, and Admin Delete.

Library SeleniumLibrary

Library OperatingSystem

Library String

Suite Setup Setup Test Environment

Suite Teardown Cleanup Test Environment

*** Variables ***

\${BROWSER} Chrome

\${BASE_URL} http://localhost:3000

\${LOGIN_URL} \${BASE_URL}/login

\${REPORT_LOST_URL} \${BASE_URL}/report-lost

\${REPORT_FOUND_URL} \${BASE_URL}/admin/report-found

\${ADMIN_DASHBOARD_URL} http://localhost:3000/admin/dashboard

\${ADMIN_ITEMS_URL} http://localhost:3000/admin/items

\${ADMIN_OWNERLIST_URL} http://localhost:3000/admin/ownerlist

--- File Upload Path ---

\${IMAGE_FILENAME} test_proof.png

\${IMAGE_PATH} \${CURDIR}\${/}\${IMAGE_FILENAME}

--- Authentication Credentials ---

\${STUDENT_EMAIL} student@gsiit.tu.ac.th

\${STUDENT_PASSWORD} Burger5\$

\${ADMIN_EMAIL} admin@gsiit.tu.ac.th

\${ADMIN_PASSWORD} Password123!

--- Static Data ---

\${STUDENT_ID_VAL} 9999999999

\${DATETIME_LOST} 2025-11-18T14:30

\${CLAIMANT_NAME} Test Claimant

\${CLAIMANT_PHONE} 0812345678

--- Locators ---

\${LOGIN_FORM} xpath=//form

\${MENU_BUTTON} xpath=//button[@aria-label="Menu"]

\${LOGOUT_BUTTON} xpath=//button[contains(@class, "logout-button")]

\${EMAIL_INPUT} name=username

\${PASSWORD_INPUT} name=password

Forms

`${LOST_SUBMIT_BUTTON}    xpath=//button[@type="submit" and text()="Submit Report"]`
`${FOUND_SUBMIT_BUTTON}    xpath=//button[@type="submit" and text()="Submit Report"]`

Identification / Mark Returned Locators

`${MARK_RETURNED_BUTTON}    xpath=//button[contains(text(), "Mark Claimed/Returned")]`
`${ID_NAME_INPUT}            xpath=//input[@placeholder="Owner's name"]`
`${ID_RADIO_STUDENT}        xpath=//input[@type="radio" and @value="student"]`
`${ID_STUDENT_ID_INPUT}    xpath=//input[contains(@class, "student-id-input")]`
`${ID_TEL_INPUT}            xpath=//input[@placeholder="Phone number"]`
`${ID_FILE_INPUT}            xpath=//input[@type="file"]`
`${ID_SUBMIT_BUTTON}        xpath=//button[text()="Submit"]`
`${CONFIRM_YES_BUTTON}     xpath=//button[text()="Yes"]`

Delete Button Locator

`${DELETE_BUTTON}            xpath=//button[text()="Delete"]`

*** Test Cases ***

Test Case 1: Student Reporting Process (Report -> View List -> View Detail)

[Documentation] Verifies the Student's ability to report a lost item and view it.

[Setup] Generate Unique Item Names

1. Login

Login As Student

2. Test Reporting Process

Verify Student Can Report Lost Item `${DYNAMIC_LOST_ITEM}`

3. Test List View Visibility

Wait Until Location Is `${BASE_URL}/home`

Wait Until Page Contains All Item List

4. Test Item Detail Navigation

Verify Item Is Listed And Click `${DYNAMIC_LOST_ITEM}`

5. Test Detail Content

Verify Item Details Page Content `${DYNAMIC_LOST_ITEM}` SIIT Main Building

6. Logout

Logout User

Test Case 2: Admin Resolution Process (Report -> View Detail -> Process Return -> Verify Owner List)

[Documentation] Verifies the Admin's ability to report found items and process a return.

[Setup] Generate Unique Item Names

1. Login

Login As Admin

2. Test Reporting Process

Verify Admin Can Report Found Item \${DYNAMIC_FOUND_ITEM}

3. Test List View Navigation

Go To \${ADMIN_ITEMS_URL}

Wait Until Page Contains All Item List

4. Test Item Detail Navigation

Verify Item Is Listed And Click \${DYNAMIC_FOUND_ITEM}

5. Test Detail Content

Verify Item Details Page Content \${DYNAMIC_FOUND_ITEM} SIIT Main Building

6. Test Return/Identification Process

Execute Admin Return Process \${CLAIMANT_NAME}

7. Test Owner List Verification

Verify Owner Verification List \${CLAIMANT_NAME}

8. Logout

Logout User

Test Case 3: Admin Delete Item Process (Report -> Delete -> Verify Removal)

[Documentation] Verifies the Admin can delete an item and that it disappears from the list.

[Setup] Generate Unique Item Names

1. Login

Login As Admin

2. Report a specific item to be deleted

Verify Admin Can Report Found Item \${DYNAMIC_DELETE_ITEM}

3. Go to List and Verify it exists

Go To \${ADMIN_ITEMS_URL}

Verify Item Is Listed And Click \${DYNAMIC_DELETE_ITEM}

4. Execute Delete Process (Handle Confirmations)

Execute Admin Delete Process

5. Verify Item is Gone

Verify Item Is Deleted From List \${DYNAMIC_DELETE_ITEM}

6. Logout

Logout User

*** Keywords ***

Setup Test Environment

Set Selenium Implicit Wait 10s

Create File \${IMAGE_PATH} dummy content for image upload test

Cleanup Test Environment

Close All Browsers

Remove File \${IMAGE_PATH}

Generate Unique Item Names

[Documentation] Creates unique item names for every test run to avoid duplicates.

\${RANDOM_ID}= Generate Random String 5 [NUMBERS]

Set Suite Variable \${DYNAMIC_LOST_ITEM} Blue Backpack Lost (Run \${RANDOM_ID})

Set Suite Variable \${DYNAMIC_FOUND_ITEM} Blue Backpack Found (Run \${RANDOM_ID})

Set Suite Variable \${DYNAMIC_DELETE_ITEM} Delete Test Item (Run \${RANDOM_ID})

--- AUTHENTICATION PROCESSES ---

Login As Student

Open Browser \${LOGIN_URL} \${BROWSER}

Maximize Browser Window

Delete All Cookies

Wait Until Element Is Visible \${EMAIL_INPUT}

Input Text \${EMAIL_INPUT} \${STUDENT_EMAIL}

Input Text \${PASSWORD_INPUT} \${STUDENT_PASSWORD}

Submit Form \${LOGIN_FORM}

Wait Until Location Is \${BASE_URL}/home

Login As Admin

Open Browser \${LOGIN_URL} \${BROWSER}

Maximize Browser Window

Delete All Cookies

Wait Until Element Is Visible \${EMAIL_INPUT}

Input Text \${EMAIL_INPUT} \${ADMIN_EMAIL}

Input Text \${PASSWORD_INPUT} \${ADMIN_PASSWORD}

Submit Form \${LOGIN_FORM}

Wait Until Location Is \${ADMIN_DASHBOARD_URL}

Logout User

Wait Until Element Is Visible \${MENU_BUTTON}

Click Element \${MENU_BUTTON}

Wait Until Element Is Visible \${LOGOUT_BUTTON}

Click Element \${LOGOUT_BUTTON}

Wait Until Location Is \${LOGIN_URL}

--- CORE TEST PROCESSES ---

Verify Student Can Report Lost Item

[Arguments] \${ITEM_NAME}

Go To \${REPORT_LOST_URL}

Wait Until Page Contains Report Lost Item

Input Text xpath=//input[@placeholder="e.g., 6522771045"] \${STUDENT_ID_VAL}

Input Text xpath=//input[@placeholder="e.g., iPhone 13 Pro"] \${ITEM_NAME}

Select From List By Value xpath=(//select[@class="select-input"])[1] Accessories

Input Text xpath=//input[@type="datetime-local"] \${DATETIME_LOST}

Sleep 0.5s

Input Text xpath=//input[@placeholder="e.g., SIIT Main Building Lobby"] SIIT Main Building

Wait Until Element Is Visible \${LOST_SUBMIT_BUTTON}

Scroll Element Into View \${LOST_SUBMIT_BUTTON}

Sleep 0.5s

Click Element \${LOST_SUBMIT_BUTTON}

Alert Should Be Present Lost item reported successfully!

Verify Admin Can Report Found Item

[Arguments] \${ITEM_NAME}

Go To \${REPORT_FOUND_URL}

Wait Until Page Contains Report Found Item

Input Text xpath=//form[@class="rf-form"]//input[@placeholder="e.g., iPhone 13 Pro"]
\${ITEM_NAME}

Input Text xpath=//form[@class="rf-form"]//textarea[@class="rf-textarea"] Description test.

Select From List By Value xpath=//form[@class="rf-form"]//select[@class="rf-input rf-select"]
Accessories

Input Text xpath=//input[@type="datetime-local"] \${DATETIME_LOST}

Sleep 0.5s

Input Text xpath=//input[@placeholder="e.g., Main Cafeteria..."] SIIT Main Building

Scroll button into view to prevent interception

Wait Until Element Is Visible \${FOUND_SUBMIT_BUTTON}

Scroll Element Into View \${FOUND_SUBMIT_BUTTON}

Sleep 0.5s

Click Element \${FOUND_SUBMIT_BUTTON}

Alert Should Be Present SUCCESS! Item reported

Verify Item Is Listed And Click

[Arguments] \${ITEM_NAME_TO_CLICK}

Wait Until Page Does Not Contain Loading items... timeout=15s

\${CARD_TITLE_LOCATOR}= Set Variable xpath=//div[contains(@class, "card-title") and contains(., "\${ITEM_NAME_TO_CLICK}")]

Wait Until Element Is Visible \${CARD_TITLE_LOCATOR} timeout=10s

Scroll Element Into View \${CARD_TITLE_LOCATOR}

Click Element \${CARD_TITLE_LOCATOR}

Verify Item Details Page Content

[Arguments] \${EXPECTED_NAME} \${EXPECTED_LOCATION}

Wait Until Page Contains \${EXPECTED_NAME} Details

Wait Until Page Contains \${EXPECTED_LOCATION}

Wait Until Page Contains Date Lost/Found:

Execute Admin Return Process

[Arguments] \${CLAIMANT_NAME}

Wait Until Element Is Visible \${MARK_RETURNED_BUTTON}

Click Button \${MARK_RETURNED_BUTTON}

Wait Until Page Contains Finalize Claim / Return

Input Text \${ID_NAME_INPUT} \${CLAIMANT_NAME}

Click Element \${ID_RADIO_STUDENT}

Input Text \${ID_STUDENT_ID_INPUT} \${STUDENT_ID_VAL}

Input Text \${ID_TEL_INPUT} \${CLAIMANT_PHONE}

Choose File \${ID_FILE_INPUT} \${IMAGE_PATH}

Scroll submit button into view

Scroll Element Into View \${ID_SUBMIT_BUTTON}

Click Button \${ID_SUBMIT_BUTTON}

Wait Until Page Contains Confirm marking item

Click Button \${CONFIRM_YES_BUTTON}

Sleep 1s

Handle Alert action=ACCEPT

Wait Until Location Is \${ADMIN_ITEMS_URL} timeout=10s

Verify Owner Verification List

[Arguments] \${CLAIMANT_NAME}

Go To \${ADMIN_OWNERLIST_URL}

Wait Until Page Contains Owner Verification Lists

Wait Until Page Does Not Contain Loading requests...

Wait Until Page Contains \${CLAIMANT_NAME}

Wait Until Page Contains \${STUDENT_ID_VAL}

\${REVIEW_BTN}= Set Variable xpath=//div[contains(@class, "owner-claimant-name") and contains(text(), "\${CLAIMANT_NAME}")]/following-sibling::div[@class="owner-action-row"]/button

Scroll Element Into View \${REVIEW_BTN}

Click Element \${REVIEW_BTN}

Wait Until Page Contains Review Verification Request

Wait Until Page Contains \${CLAIMANT_NAME}

Click Button xpath=//button[contains(text(), "Close Review")]

Execute Admin Delete Process

[Documentation] Clicks Delete, handles the 'Are you sure?' alert, and the 'Success' alert.

Wait Until Element Is Visible \${DELETE_BUTTON}

Click Button \${DELETE_BUTTON}

1. Handle the Confirmation Dialog ("Are you sure?")

Handle Alert action=ACCEPT

2. Wait for and Handle the Success Alert

Sleep 1s

Handle Alert action=ACCEPT

3. Verify redirection back to list

Wait Until Location Is \${ADMIN_ITEMS_URL} timeout=10s

Verify Item Is Deleted From List

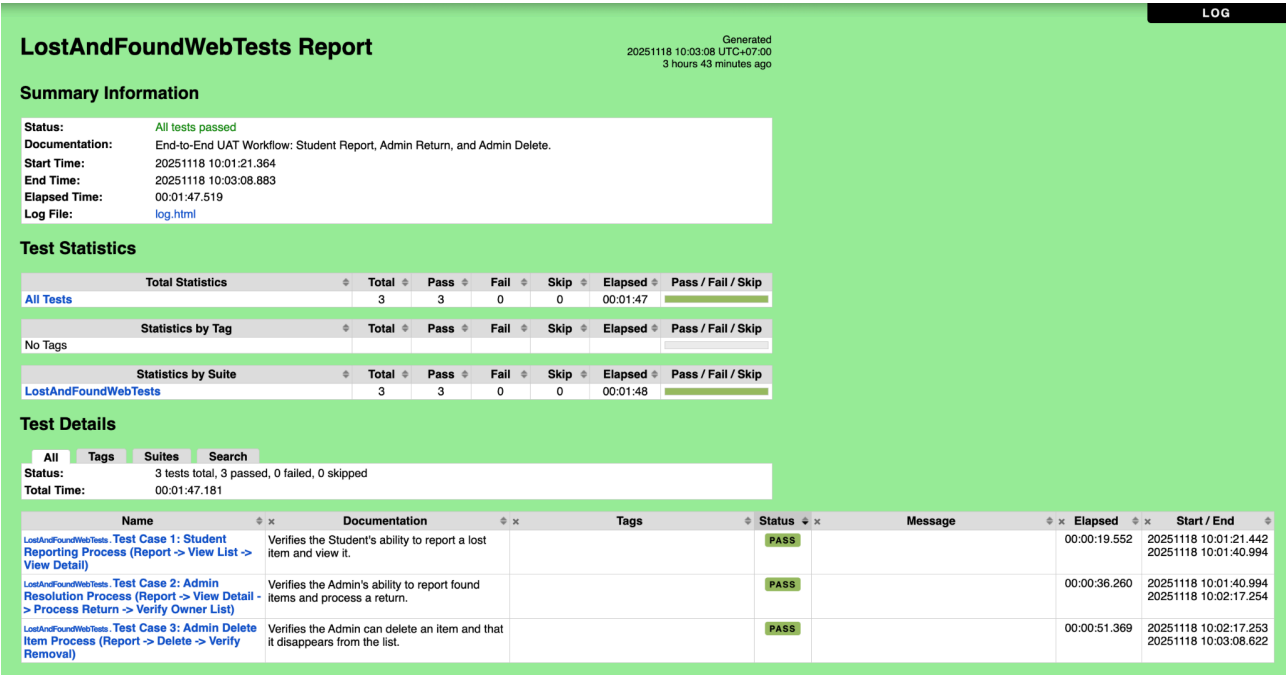
[Arguments] \${ITEM_NAME}

[Documentation] Ensures the item card is NO LONGER visible on the list page.

Wait Until Page Does Not Contain Loading items...

Page Should Not Contain \${ITEM_NAME}

Test Results



Links and Repository

- GitHub

<https://github.com/natthidathe/Campus-Lost-and-Found-System.git>
- Jira

<https://des424-g7.atlassian.net/jira/software/projects/CLFS/boards/2>
- Website

<https://main.d1bs3qseefd2a.amplifyapp.com/>